Predictive Analytics

Introduction to Applied Statistics Using R (STAT 401)

Steven Pappas

2026-10-06

Outline

1. Introduction
2. Bias Variance Trade-Off
3. Supervised vs Unsupervised Learning
4. Cross-Validation
5. Applications With Predictive Models
    - 5.1 Ridge and Lasso Regression
    - 5.2 Gradient Boosted
    - 5.3 Tree-Based Methods

Introduction

- With regression methods like simple or multiple linear regression, we are interested in **inference**
  - Interpreting relationships between one or more explanatory (X_j) variables and a response (Y)
- With predictive methods, we are interested in **minimizing** predicted error in our predicted $\hat{Y}_i$
  - Linear regression assumptions and multicollinearity are irrelevant!
  - How do we replicate predictive success with ordinarily hazardous models?

Supervised vs Unsupervised Learning

- In unsupervised learning, the set of explanatory variables (X_j) **do not** observe the response (Y)
  - Grouping observations by similarities in their explanatory matrix
  - Principal component analysis (dimension reduction) and clustering algorithms
- In supervised learning, the set of explanatory variables (X_j) observe the response (Y)
  - Any type of regression model
  - The predictive models I will cover today!

Supervised Learning

- We can think of a data set to model in terms of Y and X matrices.

$$Y_{n\times 1} \,\&\, X_{n\times p}$$

- n: Number of observations
- p: Number of predictors

$$Y_{n\times 1} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, X_{n\times p} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,p} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n,1} & x_{n,2} & \cdots & x_{n,p} \end{bmatrix}$$

- In predictive modeling, we are trying to minimize error in $\hat{Y}_i$ using our explanatory set X_j

Variable Selection

- Predictive models often use high-dimensional data sets (p is very large)
- In these cases, it's often necessary to perform **variable selection** to account for quality in variation explained in the response Y
- When interested in *inference*, variable selection procedures keep models simple but effective
- When interested in *prediction*, variable selection procedures improve repeated predictive success

Bias-Variance Trade-off

- **Bias**: How well is the model fitting to its given data?
- **Variance**: How well does the model adjust to new data?
- Minimizing the total error (a function of these two) is the tenet of predictive modeling

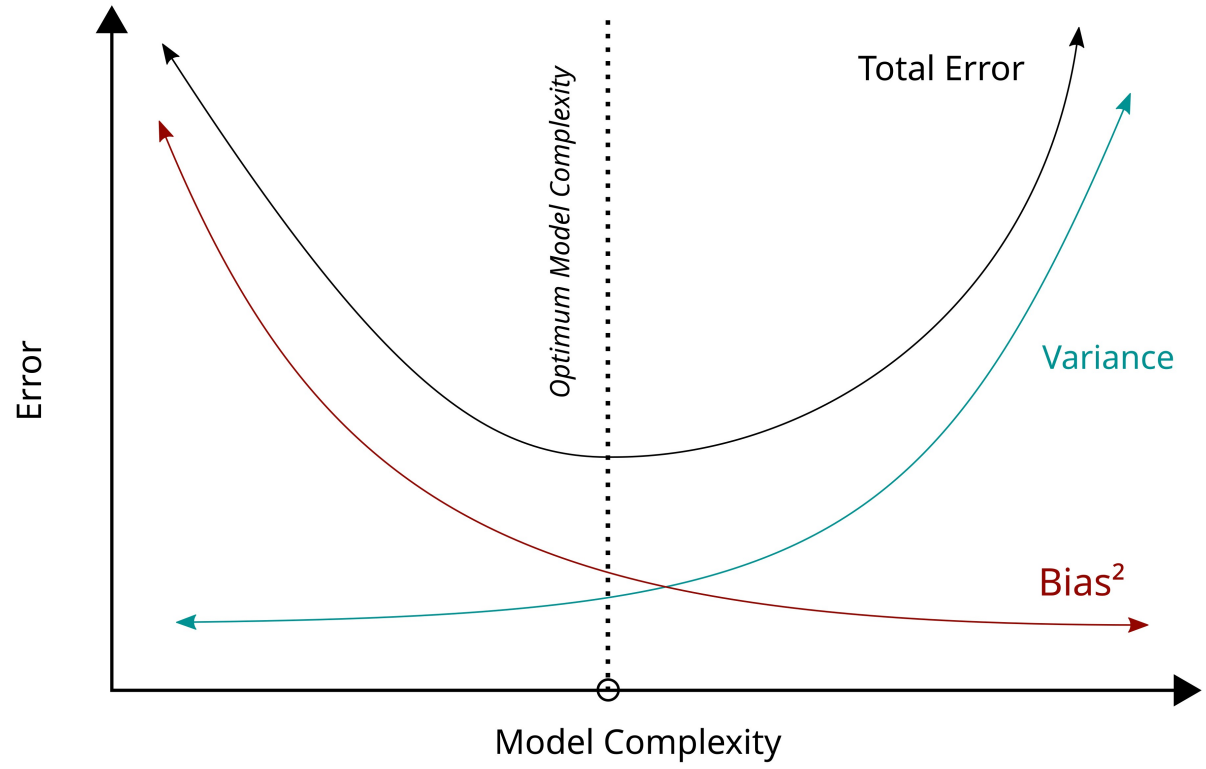


Figure 1: Bias-Variance and Total Error Curve

Cross-Validation

- Methods of ensuring consistent predictive success
- Involves splitting the data into **training** and **testing**/**validation** data sets
  - Models are built with training sets and validated with testing sets
  - Data split ensures strong, *reproducible* model results
- Let's discuss three methods of cross-validation
  1. Validation-set approach
  2. Leave-one-out cross-validation (LOOCV)
  3. k-fold cross-validation

Validation-Set Approach

- Dividing the data into one training set and testing/validation set
- Popular split is 80/20
  - 80% of the overall data goes into the training set and the other 20% is tested on
- Let's use the `createDataPartition()` function in the `caret` package

```
set.seed(1989) # For reproducability

train_index <- createDataPartition(y = boston$MEDV, p = 0.8, list = F)

nrow(boston[train_index,])/nrow(boston) # Percentage in training set
```

```
[1] 0.8043478
```

```
nrow(boston[-train_index,])/nrow(boston) # Percentage in testing set
```

```
[1] 0.1956522
```

Leave-One-Out Cross-Validation

- LOOCV trains on a single observation and tests on the rest of the data set
- Typically reserved for data sets with fewer observations to conserve computational power
- Typically minimizes *bias* with high *variance*
- Let's use the `cv.glm()` function in the `boot` package

```
set.seed(1989) # For reproducability

mod <- glm(female ~ bill_len + bill_dep + flipper_len + body_mass,
           data = penguins,
           family = "binomial")

loocv <- cv.glm(data = penguins, glmfit = mod)

loocv$delta[2] # Bias-adjusted mean squared error
```

```
[1] 0.0750726
```

k-Fold Cross-Validation

- Mix of validation set and LOOCV approaches
- Performing training and testing on k subsets of data
- Reduces *bias* without extreme *variance*
- Application using a `for` loop and the `createFolds()` function from the `caret` package to predict sex of penguins

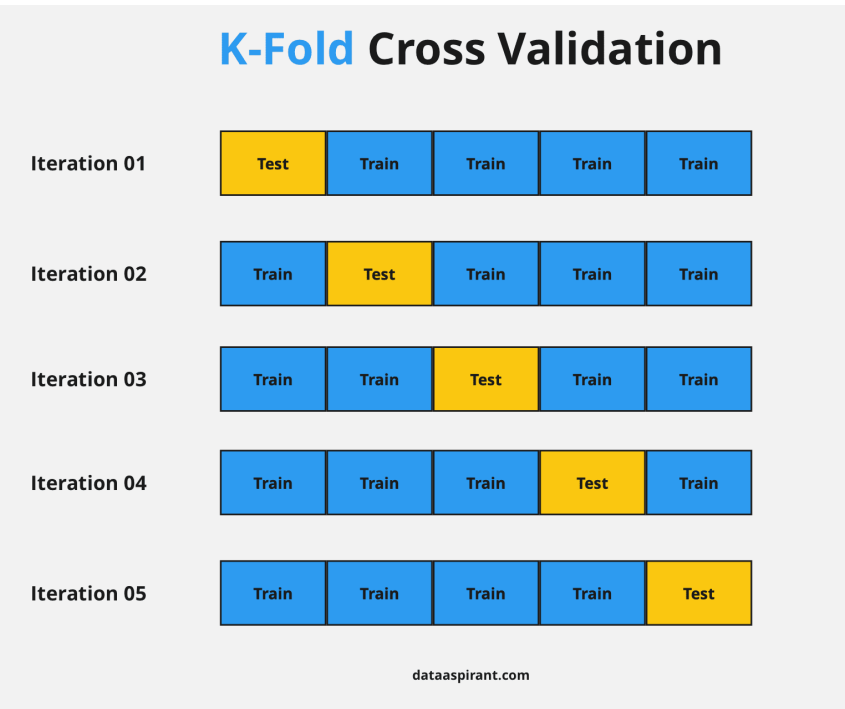


Figure 2: k-Fold Cross-Validation

k-Fold Cross-Validation

```r
set.seed(1989) # For reproducability
folds <- createFolds(penguins$female, k = 5, list = T, returnTrain = T)
fold_MSE <- numeric(5) # Dummy vector to store mean squared error

for (i in 1:5) {
  train_indices <- folds[[i]] # Each fold is a new training-testing run

  train_data <- penguins[train_indices, ]
  test_data <- penguins[-train_indices, ]

  mod <- glm(female ~ bill_len + bill_dep + flipper_len + body_mass,
             data = train_data,
             family = "binomial")

  pred <- data.frame(test_data,
                     female_hat = predict(mod, test_data, type = "response"))

  fold_MSE[i] <- mean((pred$female - pred$female_hat)^2)
}

mean(fold_MSE) # Very similar to LOOCV!
```

```
[1] 0.07639999
```

Testing Different Predictive Models

- We covered supervised learning and the importance of cross-validation and the bias-variance trade-off
- To put it all together, we will predict the median value of Boston homes in thousands of USD (Y) with 13 predictor variables ($X_1, \ldots, X_{13}$)
  - We will create models using multiple linear, ridge, and lasso regression, gradient boosting, and random forest methods
  - We will test predictive accuracy using mean-squared error and cross validate using $k = 5$ folds

Multiple Linear Regression

- Since we want to mix predictive accuracy with simplicity, we should start with a linear model
  - No variable selection procedure performed
- Vector of $\hat{\beta}$ coefficients listed below
- MSE $= 24.2$ will be our baseline for future models

```
glm_results$coef %>%
  bind_rows(.id = "fold") %>%
  group_by(var) %>%
  summarize(value = round(mean(value), 1)) %>%
  pull(var, value)
```

```
        36.5            0            0          2.7         -0.1
"(Intercept)"        "AGE"          "B"      "CHAS"       "CRIM"
        -1.5            0         -0.5        -17.8           -1
       "DIS"      "INDUS"      "LSTAT"        "NOX"    "PTRATIO"
         0.3          3.8            0            0
       "RAD"         "RM"        "TAX"         "ZN"
```

```
cat("Generalized linear model MSE:", mean(glm_results$mse))
```

```
Generalized linear model MSE: 24.20114
```

Ridge and Lasso Regression

- Ridge shrinks coefficient estimates towards zero by *squaring* the sum of the $\hat{\beta}$ coefficients
- Lasso shrinks coefficient estimates to zero by taking the *absolute value* of the sum of the $\hat{\beta}$ coefficients
  - Performs variable selection!
- Both methods require variable standardization ($z = \frac{x-\mu}{\sigma}$)

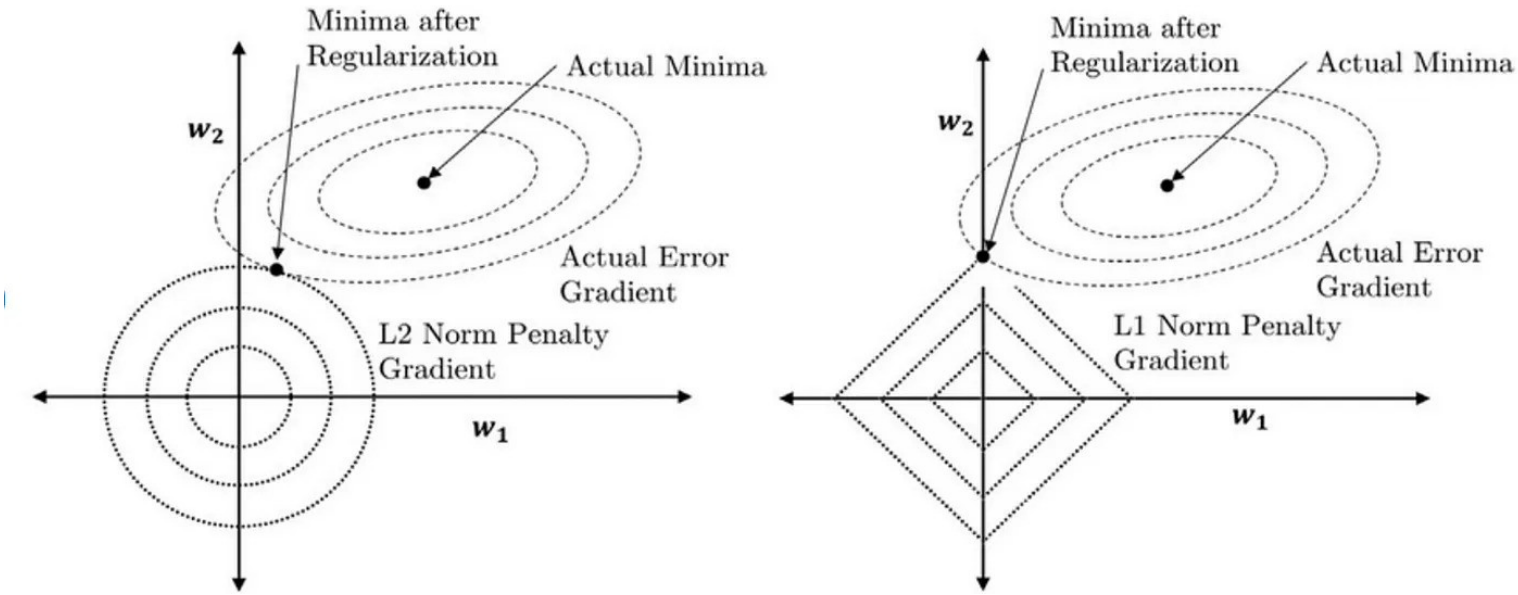


Figure 3: Ridge on Left, Lasso on Right

Ridge Regression

- Ridge regression can be performed using the `cv.glmnet()` function in the `glmnet` package
  - $\alpha = 0$ corresponds to the ridge penalty
  - This function automatically standardizes variables!
- k-fold cross-validation, and ridge procedures in corresponding R script
- MSE $= 26.9 > 24.2 \rightarrow$ worse than our generalized linear model

```
round(rowMeans(do.call(cbind, ridge_results$coef)), 1) # Model coefficients
```

```
(Intercept)        CRIM          ZN       INDUS        CHAS
       21.0        -0.1         0.0        -0.1         2.8
        NOX          RM         AGE         DIS         RAD
       -5.5         3.6         0.0        -0.5         0.0
        TAX     PTRATIO           B       LSTAT
        0.0        -0.7         0.0        -0.3
```

```
cat("Ridge Regression MSE:", round(mean(ridge_results$mse), 1))
```

```
Ridge Regression MSE: 26.9
```

Lasso Regression

- Lasso regression can also be performed using the `cv.glmnet()` function in the `glmnet` package
  - $\alpha = 1$ corresponds to the lasso penalty
  - In addition to automatic standardization, this function performs variable selection!
- k-fold cross-validation and lasso procedures in corresponding R script
- MSE $= 27.3 > 24.2 \rightarrow$ worse than our generalized linear model

```
round(rowMeans(do.call(cbind, lasso_results$coef)), 1) # Model coefficients
```

```
(Intercept)        CRIM          ZN       INDUS        CHAS
       20.4         0.0         0.0         0.0         0.0
        NOX          RM         AGE         DIS         RAD
        0.0         0.8         0.0         0.0         0.0
        TAX     PTRATIO           B       LSTAT
        0.0         0.0         0.0        -0.2
```

```
cat("Lasso Regression MSE:", round(mean(lasso_results$mse), 1))
```

```
Lasso Regression MSE: 26.2
```

Tree-Based Methods

- Creating simple splits in predictor variables (X_j) based on trends in the response (Y)
  1. Start with a **root node**, which makes the first split
  2. Subsequent splits are denoted at **decision nodes**
  3. When no more splits are made, the resulting model coefficient is called a **terminal node**
- In building decision trees, tree-based models automatically perform variable selection

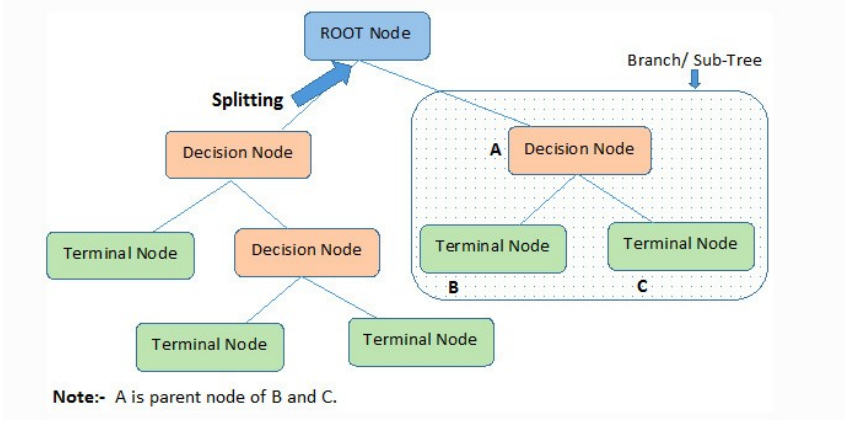


Figure 4: Tree Model Diagram

Random Forest vs Gradient Boosting

- Random forest models build decision trees on independent subsets of the data and average predictions
- Gradient boosting builds sequential models to fix previous error and stops when performance stops improving

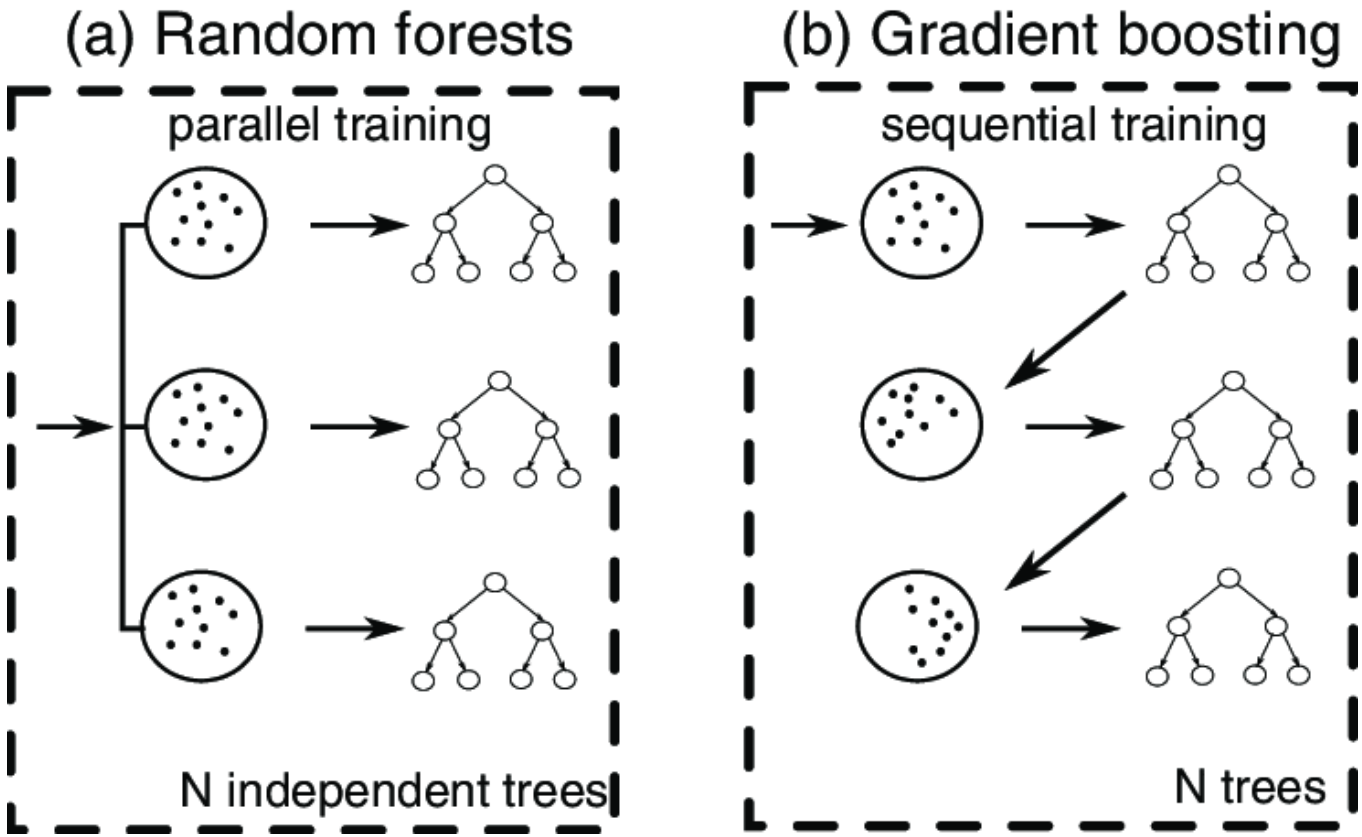


Figure 5: Random Forest vs Gradient Boosting

Random Forest Model

- Built using the `randomForest()` function in the `randomForest` package
- Model coefficients measured by **residual sums of squares** (RSS $= \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$)
  - A variable's RSS measures its variance reduction across each node split

```
rf_results$influence %>%
  bind_rows(.id = "fold") %>%
  group_by(var) %>%
  summarize(value = round(mean(IncNodePurity), 1)) %>%
  pull(var, value)
```

```
   1053.1     844.9     258.8    2148.9    2138.4    2310.2    8779.3
    "AGE"       "B"    "CHAS"    "CRIM"     "DIS"   "INDUS"   "LSTAT"
   2439.9    2314.8     407.7    8611.1    1429.6     386.8
    "NOX" "PTRATIO"     "RAD"      "RM"     "TAX"      "ZN"
```

```
cat("Random Forest MSE:", round(mean(rf_results$mse), 1))
```

```
Random Forest MSE: 11.5
```

Gradient Boosted Model

- Built using the `gbm()` function in the `gbm` package
- Model coefficients measured as **relative influence**
  - The amount of error each predictor variable reduced in subsequent boosted models

```
gbm_results$influence %>%
  bind_rows(.id = "fold") %>%
  group_by(var) %>%
  summarize(value = round(mean(rel.inf), 1)) %>%
  pull(var, value)
```

```
     0      0.1      0.6        3      1.3      0.1     49.8
 "AGE"      "B"   "CHAS"   "CRIM"    "DIS"  "INDUS"  "LSTAT"
   3.5      2.5        0     38.8      0.3        0
 "NOX" "PTRATIO"    "RAD"     "RM"    "TAX"     "ZN"
```

```
cat("Gradient Boosted MSE:", round(mean(gbm_results$mse), 1))
```

```
Gradient Boosted MSE: 7.7
```

Other Important Topics

- **Deep Learning**: Complex models built using neural networks (degrees of connected nodes)
  - Used in image classification and large language models (LLMs)
- **Support Vector Machines (SVM)**: Classification models that split groups of data in three-dimensional space using hyperplanes

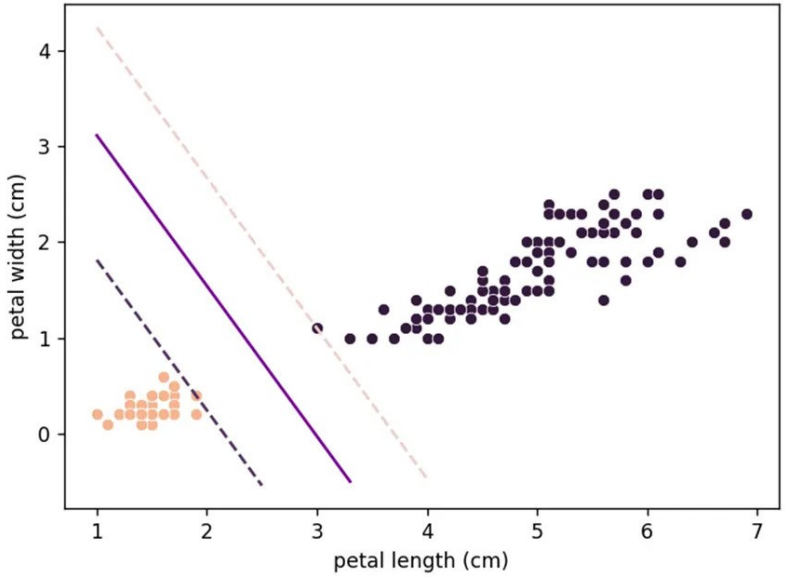


Figure 6: SVM Splitting Species in Iris Data

Thank You!

- Please visit the corresponding R script with more in-depth information about each topic
- Any questions? :)